Point Cloud Classification Using PointNet & PointNet++

Jiayi Dai, Yiyang Fu, Weiran Wang
Department of ECE & MIE
University of Toronto
{jiayi.dai, yiyang.fu, weiran.wang}@mail.utoronto.ca

Project Repository
https://github.com/weiran0129/Point-Cloud-Classification-Using-PointNet.git

Abstract

This project investigates how data augmentation and feature engineering influence PointNet and PointNet++ on ModelNet40. We implement both on-the-fly and offline augmentation pipelines, incorporate a range of geometric descriptors, and modernize the training procedure. Systematic ablations show that geometric features and simple perturbation-based augmentations substantially boost PointNet, whereas PointNet++ benefits more from jitter, dropout, scaling, normals, and improved optimizer/scheduler settings due to its deeper hierarchical nature and sensitivity to regularization and training dynamics. Rotation and flipping augmentation consistently harms performance, and excessive engineered features offer diminishing returns. These findings underscore the importance of architecture-aware augmentation and training design for point-cloud classification.

Assertion of Teamwork

Weiran Wang: Performed On-the-Fly & Offline Data Augmentation and Feature Engineering with PointNet++; Modernized training optimization pipeline; Tailored down the GitHub repository.
Yiyang Fu: Designed and implemented codes for the data augmentation and feature engineering techniques. Performed ablation tests of data augmentation and feature engineering on PointNet.
Jiayi Dai: Led the design, structure, and formatting of the project report and presentation. Conducted literature review and research on related works such as recent developments in point-based NN.

1 Introduction

Point cloud classification is an important task in 3D computer vision. Unlike images or voxels, point clouds are unordered and irregular, so standard convolutional networks do not apply directly. PointNet and PointNet++ address these challenges by learning directly from raw point sets while enforcing permutation invariance. PointNet uses shared MLPs and max pooling to extract global features (1), whereas PointNet++ adds hierarchical set abstraction to capture local geometry and handle non-uniform sampling (2). In this project, we implement both models on the ModelNet40 dataset (3), apply data augmentation and feature engineering, and compare their accuracy, robustness, and training behavior.

2 Preliminaries and Problem Formulation

2.1 Background

A point cloud is a set of 3D points optionally with additional per-point features such as normals or handcrafted geometric descriptors. Since point clouds are unordered and have no regular grid structure, learning algorithms must be permutation-invariant and able to capture both local and global geometric patterns.

2.2 Problem Formulation

Given a point cloud with coordinates and optional features, the goal is to classify it into one of 40 categories in ModelNet40 (3). We train the model by minimizing the cross-entropy loss between predicted labels and ground truth. The main objectives of this project are:

- Implement baseline PointNet and PointNet++ for ModelNet40 classification (1; 2) and compare accuracy, efficiency, and training behavior of both models.
- Evaluate how data augmentation influences robustness and whether additional geometric features improve performance.

3 Design

3.1 Dataset

We use the resampled ModelNet40 dataset `modelnet40_normal_resampled` (5) with normals. This version contains 12,311 point clouds from 40 object categories such as chairs and tables. Each object is stored as a point cloud where each point has six channels $[x, y, z, n_x, n_y, n_z]$, representing 3D coordinates and surface normals. In our experiments, we follow the standard split provided with the dataset and use 1,024 points per object as input to PointNet and PointNet++.

3.2 PointNet

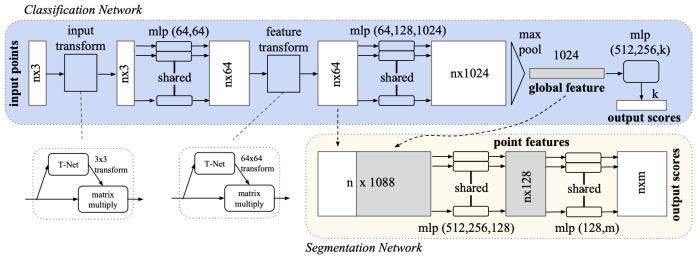

Figure 1: PointNet Architecture (1)

PointNet processes a point cloud directly as an unordered set of 3D points and enforces permutation invariance through shared MLPs and max pooling (1). The network consists of three main stages:

- **Input transformation (T-Net)**: predicts a 3×3 affine transformation to roughly align the input cloud.
- **Shared point-wise MLPs and feature transformation**: applies shared MLPs to each point and uses a second T-Net to align intermediate features.
- **Symmetric aggregation and classification**: applies max pooling over points to obtain a global feature, then uses fully connected layers and softmax for classification.

This design yields a compact, permutation-invariant representation that is robust to small perturbations and missing points, but it does not explicitly encode local structure.

3.3 PointNet++

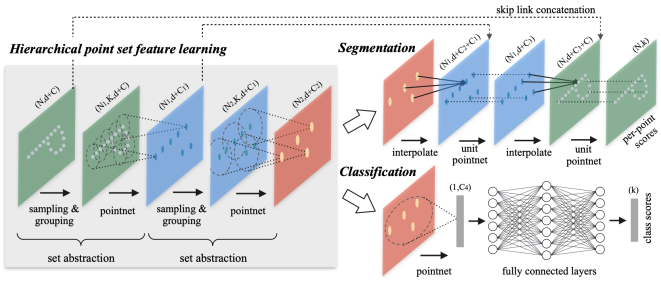

Figure 2: PointNet++ Architecture (2)

PointNet++ extends PointNet by introducing hierarchical Set Abstraction (SA) layers that capture local neighborhoods and multi-scale context (2). Each SA layer:

- Uses farthest point sampling (FPS) to select centroid points,
- Applies a ball query to group neighboring points around each centroid,
- Runs a small PointNet on each group to extract a local feature.

By stacking SA layers with fewer centroids and larger radii, PointNet++ builds features from local to global scales. In this work, we use the single-scale grouping (SSG) variant and obtain a final global feature by pooling over the last centroids and feeding it into fully connected layers for classification. This hierarchical design allows PointNet++ to better model local geometry and handle non-uniform sampling compared to PointNet.

4 Methodology

4.1 Data Augmentation

Point Clouds vary significantly in orientation, density, sampling noise and scale. To improve generalization and reduce overfitting on ModelNet40/10, we implemented geometric augmentation to the input point clouds including `Scale`, `Rotation`, `Shift`, `Flipping`, along with noise injection methods including `Jittering` and `Dropout` (See Appendix for detail augmentation formulation). Two augmentation strategies were employed on our training dataset, with the test dataset left unmodified:

On-the-Fly Augmentation (Dynamic, Per-Batch) This setting applies augmentation to each training batch iteration during the forward pass, but no new augmented files are ever written to disk.

- The dataset size remains unchanged while the model sees a different geometric variation of each point cloud every epoch.
- The unlimited variability across epochs allow model see the same sample data but arrives in slightly different randomized form (e.g. rotated, jittered, or shifted differently).
- Over many epochs, the model effectively trains on thousands of geometric variants of each object, even though only one file exists on disk.

Offline Augmentation (Static, Pre-Generated Dataset) Offline Augmentation expands the dataset by creating augmented copies of 50% of the original samples, svaing in the local disk.

- Different augmentation combinations (e.g., rotation + jitter, scale + dropout + shift, etc) are applied to a subset of the original samples.
- Offline dataset contains the original point clouds and augmented counterparts.
- Loaded by `ModelNetLoader` module prior to training, these samples do not change between epochs; instead, they provide a larger and more diverse static training corpus.

4.2 Feature Engineering

We further test if richer input representation allows PointNet/PointNet++ to better capture local surface structure and global shape properties while still maintaining the original spatial information. In addition to raw `XYZ` coordinates, we consider several geometric features, including `Surface Normals`, `Height`, `Radius`, `Curvature`, `PCA`, `Density` (See Appendix for detail features' description and formulation). These features are retrieved and computed inside the module `ModelNetLoader` before any augmentation, and the full feature vector is then concatenated into a unified per-point representation.

4.3 Training Optimization

Our baseline configuration employs the Adam optimizer, a StepLR learning-rate scheduler, and standard cross-entropy loss. To improve convergence stability and enhance generalization, we refine the optimization pipeline with techniques. (1) The original StepLR scheduler reduces learning rate abruptly every 20 epochs; replace it with Cosine Annealing, gradually decays the learning rate from 0.01 to 1×10^{-5}, yielding smoother convergence. (2) We switch the optimizer from Adam to AdamW with a weight decay coefficient of 0.05, providing stronger weight regularization. (3) Label smoothing replaces the hard one-hot target vector with a softer target distribution. Instead of giving the true class a probability of 1 and all other classes 0, we reduce the confidence of the true class to $1 - \varepsilon$ and distribute the remaining ε evenly among the other $C - 1$ classes, with $\varepsilon = 0.2$.

5 Numerical Experiments

The baseline PointNet and PointNet++ models follows architecture setup described in Section 3; with only raw XYZ coordinates as input, with no data augmentation and no additional geometric features. We then evaluate their performance under various augmentation strategies, feature-engineering settings, and improved optimization techniques described in Methodologies, comparing both instance accuracy and mean per-class accuracy. Unless otherwise specified, all experiments use the default training setup with epochs of 50, batch size of 24, and an initial learning rate of 0.001, and are trained on GPU hardware (NVIDIA T4 / RTX).

Table 1 and Table 2 summarize the selected cases of PointNet and PointeNet++ using On-the-Fly data augmentation, respectively, while Table 3 presents offline data augentation results on ModelNet10 (Due to computational constraints: generating augmented copies requires additional disk space and preprocessing time, and training on a substantially larger offline-expanded dataset as in ModelNet40 is infeasible under our storage and compute limitations).

Table 1: PointNet Test Accuracy on ModelNet40 (On-the-Fly Data Augmentation)

Configuration	Inst. Acc. (%)	Class Acc. (%)	ΔInst.	ΔClass
PointNet Baseline	87.94	82.73	0.0	0.0
Dropout	88.87	83.24	+0.93	+0.51
Jitter	88.71	83.58	+0.77	+0.85
Dropout + jitter	89.07	84.15	+1.13	+1.42
Dropout + jitter + normals	90.65	87.71	+2.70	+4.98
Dropout + jitter + normals + height + radius	91.69	88.41	+3.75	+5.67

Table 2: PointNet++ Test Accuracy on ModelNet40 (On-the-Fly Data Augmentation)

Configuration	Inst. Acc. (%)	Class Acc. (%)	ΔInst.	ΔClass
PointNet++ Baseline	91.39	88.26	0.0	0.0
Rotation	89.48	87.85	-1.91	-0.41
Flip	90.62	88.21	-0.77	-0.05
Dropout	91.72	88.16	+0.33	-0.10
Jitter (σ=0.01, clip $= 0.05$)	91.81	88.88	+0.42	+0.62
Shift & Scale	91.84	88.68	+0.45	+0.42
Dropout+Jitter+Shift+Scale	90.91	87.93	-0.48	-0.33
Dropout+Jitter+Normals	92.14	89.28	+0.75	+1.02
Radius	91.18	88.71	-0.21	+0.45
Dropout+Jitter+Height+Radius+Normals	91.89	89.24	+0.50	+0.98
Dropout+Jitter+Height+Normals	91.96	88.60	+0.57	+0.34
Dropout+Jitter+Normals (with optimization)	92.68	90.14	+1.29	+1.88
Dropout+Scale+Jitter+Normals (with optimization & 150 epochs)	93.46	91.10	+2.07	+2.84

Table 3: PointNet++ Test Accuracy on ModelNet10 (Offline Data Augmentation)

Configuration	Inst. Acc. (%)	Class Acc. (%)	ΔInst.	ΔClass
PointNet++ Baseline	92.87	92.60	0.0	0.0
Flip	91.99	91.81	-0.88	-0.79
Scale	92.43	92.05	-0.44	-0.55
Rotation	92.54	92.16	-0.33	-0.44
Shift	92.98	93.13	+0.11	+0.53
Jitter (σ=0.01, clip $= 0.05$)	93.20	93.02	+0.33	+0.42
Dropout	93.42	92.84	+0.55	+0.24
Dropout+Scale+Shift+Jitter (denoted as $[*]$)	93.64	93.21	+0.77	+0.61
$[*]$+Normals+Height	93.75	93.70	+0.88	+1.10
$[*]$+Normals+Density+PCA	94.19	93.89	+1.32	+1.29
$[*]$+Normals+Curvature	94.52	94.40	+1.65	+1.80
$[*]$+Normals+Curvature+Density+Radius	94.52	94.51	+1.65	+1.91

See complete experiment results and logs in our GitHub Repository and Appendix if interested.

6 Results & Discussion

PointNet v.s. PointNet++ The study confirms that PointNet++ consistently outperforms PointNet, though at the cost of substantially longer training time and higher computational requirements due to its hierarchical design. Our ablation results also show that the two architectures react differently to augmentation and engineered features. PointNet benefits far more from these techniques because its global max-pooling limits its ability to capture local structure; adding normals, height, or radius yields notable improvements of up to +3.8 pp. In contrast, PointNet++ already learns rich local descriptors, so engineered features provide only small gains, and in some cases excessive features introduce noise rather than useful information.

Improvements from Augmentation, Feature Engineering, and Optimization Across our experiments, the most reliable improvements come from augmentations and features that enrich geometric information without distorting the object's global structure. Surface normals provide the strongest single boost, giving both PointNet and PointNet++ high-quality local cues beyond raw XYZ. Similarly, jitter and point dropout offer effective noise regularization by adding mild variation while preserving original geometry, especially important in the on-the-fly setting, where the original shapes are continuously replaced by perturbed versions. Overall, the best-performing strategies combine normals + light noise augmentations, while avoiding geometry-distorting transforms.

Accuracy Reduction In the on-the-fly augmentation setting, rotation and flip consistently reduce accuracy. Because ModelNet objects are upright and canonically aligned, random orientation changes create a train–test mismatch and remove useful cues still present during evaluation. PointNet and PointNet++ also already include rotation-robust components (T-Net and symmetric pooling), so extra rotations or flips add little new invariance while distorting meaningful structure. Many categories further depend on orientation (e.g., chairs, tables), making such transforms harmful to class separability. We also find that adding too many engineered features can degrade performance—after essential cues like normals, height, and radius, additional descriptors often introduce noise under sampling variation rather than yielding monotonic improvements.
By contrast, offline augmentation yiled consisten improvements for most augmentation combinations (aside from rotation and flipping for the reasons above), due to unmodified samples remain in the dataset.

Limitation Due to computational constraints, not all combinations were fully evaluated on ModelNet40/10, and deeper architectures such as PointNet++ would likely benefit from larger training epochs when additional features are included. While offline augmentation is expected to provide stronger gains for large-scale datasets like ModelNet40, its practical cost is substantial: generating augmented copies significantly increases disk usage, preprocessing time, and overall training duration as the dataset grows. These resource requirements limited our ability to apply full offline augmentation to the ModelNet40 experiments, and represent an important consideration for future work.

Offline vs. On-the-Fly Augmentation: Tradeoffs Our two data augmentation strategies offer different tradeoffs: offline augmentation expands the dataset but requires significant preprocessing time and storage, while on-the-fly augmentation can be more practical with well-developed model architecture, providing unlimited variability with no preprocessing cost, though repeated perturbations may distort fine geometry.

7 Conclusions

In this project, we studied how data augmentation and feature engineering interact with PointNet and PointNet++ for point cloud classification. Both models benefit from added geometric features and targeted augmentations. However, rotation and flipping consistently reduced accuracy, as they destroy useful canonical pose information without providing meaningful new invariance. Overall, the most reliable gains come from a compact feature set combined with light on-the-fly augmentations such as jitter, dropout, and mild scaling. While offline augmentation can yield stronger improvements by expanding the dataset, it requires significant preprocessing and offers diminishing returns on canonical datasets like ModelNet. Future work could explore architectural modifications to PointNet-style models, such as improved local neighborhood encoding, or transformer-based attention layers, to better leverage geometric features. In addition, applying more advanced augmentation strategies and evaluating these models on less canonical, more challenging datasets would provide a stronger assessment of robustness and real-world generalization.

References

[1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 77–85.

[2] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5099–5108.

[3] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D ShapeNets: A Deep Representation for Volumetric Shapes," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 1912–1920.

[4] G. Qian, Y. Li, H. Peng, J. Mai, H. A. H. Hammoud, M. Elhoseiny, and B. Ghanem, "PointNeXt: Revisiting PointNet++ with Improved Training and Scaling Strategies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] X. Chen, "ModelNet40 Normal Resampled," Kaggle Dataset, 2020. Available at: `https://www.kaggle.com/datasets/chenxaoyu/modelnet-normal-resampled`.

Appendix

Appendix 1 - Data Augmentation Formula

For input point cloud $P = \{ p_i \}_{i=1}^{N}$, $p_i \in \mathbb{R}^3$ with coordinates $p_i = (x_i,\ y_i,\ z_i)$, we apply the following techniques.

Random point dropout Randomly discards a subset of points to simulate missing data and varying density. This improves robustness to occlusions and sensor sparsity.

$$\text{Dropout}(P;\theta) = \{p_i \in P \mid u_i > \theta\}, \quad u_i \sim \mathcal{U}(0,1),\ \theta \in [0,1).$$

Random scaling Applies a random global scaling factor to the entire point cloud. This helps the model generalize over objects/scenes of different absolute sizes.

$$s \sim \mathcal{U}(s_{\min}, s_{\max}), \quad \tilde{p}_i = s \cdot p_i, \quad \tilde{P} = \{\tilde{p}_i\}_{i=1}^{N}.$$

Shift (translation) Adds a small random translation vector to all points. This reduces sensitivity to the absolute location of the object/scene in the coordinate frame.

$$t \sim \mathcal{U}(-\delta, \delta)^3, \quad \tilde{p}_i = p_i + t, \quad \tilde{P} = \{\tilde{p}_i\}_{i=1}^{N}.$$

Rotation around Z axis Rotates the point cloud around the vertical (z) axis by a random angle.

$$\theta \sim \mathcal{U}(-\pi, \pi), \quad R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\tilde{p}_i = R_z(\theta)\, p_i, \quad \tilde{P} = \{\tilde{p}_i\}_{i=1}^{N}.$$

Jittering Adds small Gaussian noise to each point independently, often with clipping, to encourage robustness to sensor noise. The perturbation is kept small so that geometry is not drastically altered.

$$\epsilon_i \sim \mathcal{N}(0, \sigma^2 I_3), \quad \epsilon_i \leftarrow \text{clip}(\epsilon_i, -c, c),$$

$$\tilde{p}_i = p_i + \epsilon_i, \quad \tilde{P} = \{\tilde{p}_i\}_{i=1}^{N}.$$

Flipping (mirror) Randomly reflects the point cloud along one or more axes (e.g., x or y). This can simulate symmetrical viewpoints and reduce directional bias.

$$s_x = \begin{cases} -1 & \text{with prob. } 0.5 \\ 1 & \text{otherwise} \end{cases}, \quad s_y = \begin{cases} -1 & \text{with prob. } 0.5 \\ 1 & \text{otherwise} \end{cases},$$

$$S = \text{diag}(s_x, s_y, 1), \quad \tilde{p}_i = S\, p_i, \quad \tilde{P} = \{\tilde{p}_i\}_{i=1}^{N}.$$

Appendix 2 -Feature Engineering Formula

Radius feature Computes the distance of each point from a reference center (e.g., cloud centroid). This encodes how "far" a point is from the object's core, which can help capture radial structure.

$$\bar{p} = \frac{1}{N} \sum_{i=1}^{N} p_i, \quad r_i = \|p_i - \bar{p}\|_2,$$

$$\hat{r}_i = \frac{r_i}{\max_j r_j + \varepsilon}, \quad \text{radius feature } f_i^{\text{rad}} = \hat{r}_i.$$

Height feature Uses normalized height (z-coordinate) as an extra channel, often w.r.t. the floor or min/max of the scene. This is very useful in indoor scenes where vertical position correlates with semantics.

$$z_i = (p_i)_z, \quad z_{\min} = \min_i z_i, \quad z_{\max} = \max_i z_i,$$

$$h_i = \frac{z_i - z_{\min}}{z_{\max} - z_{\min} + \varepsilon}, \quad \text{height feature } f_i^{\text{h}} = h_i.$$

PCA-based local features For each point, compute PCA on its local neighborhood and derive eigenvalue-based descriptors (e.g., linearity, planarity, scattering). These encode local shape structure such as edges and surfaces.

$$\mathcal{N}_k(i) = \text{kNN}(p_i), \quad \mu_i = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i)} p_j,$$

$$X_i = [p_j - \mu_i]_{j \in \mathcal{N}_k(i)} \in \mathbb{R}^{k \times 3}, \quad C_i = \frac{1}{k} X_i^\top X_i,$$

$$\lambda_{i,1} \geq \lambda_{i,2} \geq \lambda_{i,3} \text{ eigenvalues of } C_i,$$

$$\text{linearity } L_i = \frac{\lambda_{i,1} - \lambda_{i,2}}{\lambda_{i,1}}, \quad \text{planarity } P_i = \frac{\lambda_{i,2} - \lambda_{i,3}}{\lambda_{i,1}}, \quad \text{scattering } S_i = \frac{\lambda_{i,3}}{\lambda_{i,1}}.$$

Curvature feature Defines a scalar curvature-like measure from neighborhood PCA, often using the smallest eigenvalue. High curvature corresponds to corners or highly varying regions.

$$\lambda_{i,1} \geq \lambda_{i,2} \geq \lambda_{i,3} > 0, \quad \kappa_i = \frac{\lambda_{i,3}}{\lambda_{i,1} + \lambda_{i,2} + \lambda_{i,3}}, \quad \text{curvature feature } f_i^{\text{curv}} = \kappa_i.$$

Density feature Measures how many neighbors lie within a fixed radius around each point, normalized somehow. This encodes local sampling density, which can help the network handle non-uniform point distributions.

$$\mathcal{B}_r(i) = \{p_j \in P \mid \|p_j - p_i\|_2 \leq r\}, \quad n_i = |\mathcal{B}_r(i)|,$$

$$d_i = \log(1 + n_i), \quad \hat{d}_i = \frac{d_i}{\max_j d_j + \varepsilon}, \quad \text{density feature } f_i^{\text{dens}} = \hat{d}_i.$$

Surface normals Surface normals give each point an estimated local orientation, typically as a unit vector in $\mathbb{R}^3$. They provide strong geometric cues about whether a point lies on a flat surface, edge, or corner, and are widely used in point-based deep networks as additional input channels. The surface normals information is included in the ModelNet40 dataset, we can readily use it without manually computing again.

Table 4: PointNet Test Accuracy on ModelNet40 (On-the-Fly Data Augmentation, All Tested Cases)

Configuration	Inst. Acc. (%)	Class Acc. (%)	ΔInst.	ΔClass
No Augmentation (Baseline)	87.94	82.74	0.00	0.00
Rotation along Z-axis	82.90	76.55	-5.04	-6.18
Flip	87.14	82.17	-0.81	-0.57
Shift	87.50	82.04	-0.44	-0.70
Scale	87.58	82.11	-0.36	-0.62
Dropout	88.87	83.24	+0.93	+0.51
Jitter	88.71	83.58	+0.77	+0.85
dropout + scale + shift + jitter + flip	87.06	81.25	-0.89	-1.49
drop + jitter	89.07	84.15	+1.13	+1.41
dropout + scale + shift	89.03	84.08	+1.09	+1.34
dropout + scale + shift + jitter	88.87	83.37	+0.93	+0.06
dropout + jitter + use normals	90.65	87.71	+2.70	+4.98
dropout + jitter + use normals + FPS	91.09	87.00	+3.15	+4.26
dropout + jitter + normals + height	91.45	87.64	+3.51	+4.91
dropout + jitter + normals + height + radius	91.69	88.41	+3.75	+5.67
dropout + jitter + normals + height + FPS	91.09	86.55	+3.15	+3.82
dropout + jitter + normals + height + radius + FPS	90.81	87.53	+2.86	+4.79
dropout + scale + shift + normals + height + radius	91.61	88.01	+3.67	+5.27
dropout + jitter + normals + height + radius + pca + curvature + density	91.61	88.30	+3.67	+5.56

Contest for Information Sharing

Consent for Sharing Project Materials

- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Group Identification

- Group number: 14
- Names of group members: Jiayi Dai, Yiyang Fu, Weiran Wang
- Signature of of group members: Jiayi Dai, Yiyang Fu, Weiran Wang
- Date: Dec 11, 2025